HS4002: Lecture 2B

Getting Started in Python

Ma Xiangyu

August 26, 2026

Table of Contents

Fetching lab notebooks

Fetching notebooks

First, navigate to the right folder. Then, the following:

```
git checkout main
git fetch upstream
git merge upstream/main
git push origin main
git checkout -b week2
```

Basics of Coding in Python

Installing and Loading Libraries I

- Python packages (or, libraries) are collections of code written by others.
- We've done this last week using `pip install` inside our virtual environment.
  - $\rightarrow$ By default, this fetches packages from the PyPI network.

Installing and Loading Libraries II

- Libraries specialize in different things.
  - → pandas is the main package that facilitates data collection and manipulation
  - → plotnine is a package that helps plot graphs, using a grammar of graphics
  - → openpyxl (or xlrd) is a package pandas uses under the hood to read Excel files
- After installing the libraries, you need to import them.

Example

```
import pandas as pd
```

`as pd` gives the package a short nickname, so you type `pd.read_csv(...)` instead of `pandas.read_csv(...)` every time.

Object Assignments

- We create objects in Python using =.
- Object names can only contain letters, numbers, and underscores — and can't start with a number.

Object Assignments

- Assign names that “make sense" to you.
  - → e.g. df for a data frame
- Avoid using names that might conflict with functions in your packages, or with Python’s own reserved words
  - → e.g. don’t name something `list`, `type`, or `sum` — these are existing Python functions, and reusing the name would cause weird conflicts.

Conditionals

- `if` / `elif` / `else` run different code depending on a condition.
- Indentation marks what belongs inside each branch — same rule as functions.

Example

```
age = 17
if age < 18:
    category = "minor"
elif age < 30:
    category = "young adult"
else:
    category = "adult"
```

This “if this, then that" logic is exactly what `np.where()` does later — just applied to a whole pandas column at once, instead of one value at a time.

Example: Checking a Quiz Score

```
quiz_score = 7

if quiz_score >= 8:
    message = "Ready to move on"
elif quiz_score >= 5:
    message = "Review a few topics"
else:
    message = "Ask for help"

print(message)
```

The conditions are checked from top to bottom. Python stops as soon as one of them is true.

Example: Combining Conditions

```
attendance = 0.80
submitted = True

if attendance >= 0.75 and submitted:
    status = "Requirements met"
elif not submitted:
    status = "Assignment missing"
else:
    status = "Attendance too low"
```

`and` requires both conditions to be true. You can also use `or` when either condition is enough, and `not` to reverse a condition.

Functions

- At the most abstract level, a function is a sequence of instructions that maps a set of inputs to a set of outputs.
- For the most part, you will be using the functions that others are writing.
  - → You will start writing these more as you get more experienced in Python, or develop specialized needs.
- Try to understand both (a) the input parameters, and (b) the output structure.
  - → In a Jupyter notebook, put a question mark *after* the function to find out (e.g. `pd.read_csv?`), or use `help(pd.read_csv)` anywhere.

Example

```
def example_function(some_input):
    some_output = some_input * 2
    return some_output
```

Python uses indentation (not curly braces) to mark what's inside the function — indent consistently, or you'll get an error.

What is an Object?

- Everything in Python is an **object** — numbers, strings, lists, dictionaries, and (as we're about to see) a whole DataFrame.
- An object bundles together **data** it holds and **things it can do**.
- You've actually been using objects and their dot-notation all lecture, without a name for it yet.

Attributes vs. Methods

- An **attribute** is a piece of data stored on an object. Access it with `object.attribute` — no parentheses.
- A **method** is a function that belongs to an object. Access it with `object.method()` — always has parentheses, even when empty.
- Rule of thumb: no parentheses means “just look this up"; parentheses means “go compute/do this."

Example

```
df.shape        # attribute -- just a lookup
df.columns      # attribute -- no ()

df.dropna()     # method -- does something
df.sample(n=500)  # methods can take arguments

"hello".upper()   # even a plain string has methods
```

Understanding Data Structure

The Data Frame

- The basic data we're going to be working with is the data frame.
  - → Specifically, we are working with pandas' implementation, called the `DataFrame`.
- A data frame comprises rows and columns
  - → Each individual column is a pandas `Series`.

Data Types

- There are four main data types you will come across (pandas calls these `dtypes`).
  - → `int64`: integers
  - → `float64`: numeric (or, real numbers)
  - → `object`: characters (or, strings)
  - → `datetime64`: date/time objects

Important Basic Functions to Know

How to read data

- We generally use some type of “read" function to read data.
- Data tends to come in the form of .csv, .xls, or .dta files. Use the appropriate read function for each.
  - → Such as `pd.read_csv`, or `pd.read_stata`.
- Make sure the data file is sitting in the right directory.
  - → Use `os.getcwd()` (after `import os`) if you’re unsure what directory you’re in.

Examples

```
# CSV file
df = pd.read_csv("example.csv")

# Excel file
df = pd.read_excel("example.xlsx")

# Stata file
df = pd.read_stata("example.dta")
```

Use the function that matches your file type.

How to write data

- When we're done manipulating our data, we often want to save it.
- This requires that we write the data.
- We use different methods, depending on the intended output.

  $\rightarrow$ E.g. `.to_csv()` if we want to produce a .csv file.
- It's a good idea to name the output file something different, so that you preserve the input file.

Example

```
df.to_csv("output.csv", index=False)
```

`index=False` stops pandas from writing its row-number index out as an extra column.

Data inspection functions

- To check the number of dimensions of your data frame, use `df.shape` (rows, columns).
- To just retrieve the number of rows, use `df.shape[0]` (or `len(df)`).
- To just retrieve the number of columns, use `df.shape[1]`.
- To check the data type of each column, use `df.dtypes`.
- To check all the column names, use `df.columns`.
- To print, just type `df` on its own line in a Jupyter cell (or `print(df)`).

Selecting columns

- We select specific columns to retain by indexing the data frame with a list of column names.
- Often, a data frame starts off with many data columns we're uninterested in.
- In the interest of efficiency and cleanness, we can remove them.

Example

```
df_smaller = df.loc[:, ["var1", "var2", "var3"]]
```

Selecting rows

- We use boolean indexing to narrow the data set to rows we're interested in.

Example

```
# filter for women
df_women = df.loc[df["gender"] == "woman", :]
```

Creating new columns

- Assigning to a new column name creates a column that is a function of an existing column.
- It is often used alongside `np.where` conditionals (from `import numpy as np`).
- You will likely be using this a lot to create, amend, or extend variables.

Example

```
# using np.where as a condition
# np.where takes the following structure:
# np.where(condition, output_if_true, output_if_false)
df["women_string"] = np.where(df["women"] == 1,
"Women", "Not women")
```

Change column types

- One common type of column change might do is called “casting," when we change the type of a column from one type to another.
- Frequently, we may need to re-code column that has been interpreted as “numeric" into a category, for instance.

Example

```
# this adds a string version of the women column
df["women_binary"] = df["women"].astype(str)
```

Sampling

- We use `.sample()` to take a random sample of a data frame.
- It's often a good idea to set a seed, using the `random_state` parameter, so that the randomization is controlled.

Example

```
# take a random sample of 500
# random_state fixes the seed -- can be any
# number, I like 47
df_sample = df.sample(n=500, random_state=47)
```

Troubleshooting

How to troubleshoot problems

- It's important to learn what each function does.
  - → In a Jupyter notebook, put a question mark *after* the command to learn how it works.
  - → E.g. `pd.read_stata?` to learn what the `read_stata` function does. Outside Jupyter, use `help(pd.read_stata)` instead.
- Learning how to troubleshoot problems is the single most important skill you have to pick up.
  - → Google the error messages you receive.
  - → This is also literally what AI is best at ATM.

How to use AI for now

- We'll be using AI in more integrated/sophisticated ways as we get further down the semester.
- For now, I'd advise you to talk to chatbots when you're stuck on any programming task.
  - $\rightarrow$ Tell it what you're trying to do, the language you're writing in, and perhaps the exact error message or code you're using.
  - $\rightarrow$ AI should be excellent at correcting issues you run into.

Making Tables with pandas

Building your own data frame

- You don't need to read a file to get a data frame — you can create and edit your own tables directly in pandas.
- There are two common ways to construct one by hand.

Example

```
# you can create data frames in two ways:
# first way -- a dictionary of columns
df1 = pd.DataFrame({"person": ["John", "Jill"],
                    "height": [1.73, 1.71]})
# second way -- a list of rows
df2 = pd.DataFrame([("John", 1.73),
                    ("Jill", 1.71)],
                   columns=["person", "height"])
```

Example

```
# this assigns a height of 2.0
# to the person named John
df2.loc[df2["person"] == "John", "height"] = 2.0
```

What is `to_latex`?

- `.to_latex()` is a simple method (built directly into pandas) that we will be using to generate slightly prettier tables.
  - → You can read more about it here: `https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_latex.html`
- pandas can also produce tables in other formats, such as `.to_html()` and `.to_markdown()`.
- You can customize column names, captions, decimal points and so on.

Example

```
df2.to_latex(
    index=False,
    float_format="%.1f",
    caption="Test Table",
    header=["Person", "Height"])
```

Making Graphs with plotnine

What is plotnine?

- plotnine is a Python package for producing statistical graphics.
- It has a simple underlying grammar.
- You provide plotnine with (a) the data, (b) instructions on what figures to draw using the data, and (c) how to customize and refine the graphic.
- A guide on how to build these graphics: https://plotnine.org

plotnine structure

1. Layer(s) of geoms
   → These are the “graphs" we draw.
2. Scales and mappings
   → How we map data to values in the aesthetic space (e.g. women = red, men = blue)
3. Coordinates
   → We’re mostly using cartesian coordinates (cf. map projections, polar coordinates etc.)
4. Facets
   → For dividing the data into subsets
5. Themes
   → For controlling other elements of display, such as font or background color.

Thanks for listening :)